Ejercicios del Tema 2 (2/3)

Tras haber leído y comprendido la sección 4 del tema 2, resuelva estos ejercicios.

Ejercicio 1

Escriba una clase que permita crear objetos de tipo `Point`. Cada objeto de tipo Point estará caracterizado por sus coordenadas `(x,y)` todas de tipo entero. Además, un `Point` sabrá determinar su distancia a otro `Point` utilizando la función M`ath.sqrt()`.

```
Point p1 = new Point(100,50);
Point p2 = new Point(100,60);

double d = p1.distance(p2);
```

Ejercicio 2

Escriba una clase que permita crear objetos de tipo `Circle`. Cada objeto de tipo `Circle` estará caracterizado por el punto central del círculo `(x,y)` y su radio `r` todo de tipo entero. Además, un `Circle` sabrá determinar si intersecta o no con otro `Circle`.

```
Circle c1 = new Circle(100,50,20);
Circle c2 = new Circle(100,60,20);

boolean intersect = c1.intersects(c2);
```

Ejercicio 3

Escriba una clase que permita crear objetos de tipo `Rectangle`. Cada objeto de tipo Rectangle estará caracterizado por el punto superior izquierdo `p1` e inferior derecho `p2`. Además, un `Rectangle` sabrá determinar si intersecta o no con otro `Rectangle` devolviendo como resultado el Rectangle de interseccion.

```
Rectangle r1 = new Rectangle(100,50,120,60);
Rectangle r2 = new Rectangle(100,60,200,80);

Rectangle r3 = r1.intersects(r2);
```

Ejercicio 4

Escriba una clase que permita crear objetos de tipo `ComplexNumber`. Cada objeto de tipo ComplexNumber estaŕa caracterizado por su arte real y su parte imaginaria. Además, un ComplexNumber permitirá realizar la suma y el producto con otro `ComplexNumber`. Discutir si es necesaria la resta.

```
ComplexNumber n1 = new  ComplexNumber(5,3); // 5 + 3i
ComplexNumber n2 = new  ComplexNumber(3,2); // 5 + 3i
ComplexNumber n3 = n1.add(n2);
ComplexNumber n4 = n1.by(n2);
System.out.print(n3.getRealPart());
System.out.print("+");
System.out.print(n3.getImaginaryPart());
System.out.println("i");
```

Ejercicio 5

Escriba una clase que permita crear objetos de tipo `IntegerLinkedStack`.

Esta clase implementará una pila de enteros (Stack) con las operaciones habituales (push, pop e isEmpty) utilizando internamente una estructura de nodos enlazados que contengan los valores enteros a almacenar.

El siguiente ejemplo ilustra un caso de uso de la estructura de datos solicitada.

```
IntegerLinkedStack s = new IntegerLinkedStack();
s.push(23);
int e = s.pop();
boolean b = s.isEmpty();
```